

Training System — Joel's Complete Onboarding Guide

Scoring System · Simulator · QA · CI/CD · Everything from Scratch

Antonio Hermoso — Technical Director

April 29, 2026

Welcome, Joel

This document is your single source of truth. Everything you need to build your piece of the **Training** system — the scoring simulator, the E2E tests, the seed data, and the CI/CD pipeline — is here.

Read it in one sitting. It's long, but it's written for you specifically. Don't skip sections — each one builds on the previous.

CRITICAL NOTE — READ THIS FIRST: Some earlier docs (including parts of `joel-en.md`) describe the stack as TypeScript + Node + React. **That plan was scrapped.** The actual code in the repository is **Python 3.12 + Flask 3**. If you see references to `.ts` files, `pnpm`, `Vite`, `Prisma`, or `ScoringSimulator.tsx` in other docs, **ignore them**. This document describes reality. When in doubt: **code wins over documents**.

The Project in 5 Minutes

What is Training?

Training is an automated competitive evaluation system for fire-truck driver candidates applying to CMadrid (Madrid Region Fire Department). It is a Spanish public competitive exam — an *oposición pública*.

The system:

- Captures sensor data from the Doback Elite device installed in each fire truck
- Also receives fleet data from Webfleet (Bridgestone's external platform that CMadrid has contracted)
- Scores each candidate's driving attempt on a 0–10 scale
- Maintains a daily-updated ranking of all candidates
- Closes formally with a legally binding document (acta) signed by two administrators

Roughly 265 candidates compete per cohort (*convocatoria*) for a fixed number of spots (*plazas*).

The critical difference: competition, not exam

This is NOT a driving license test where you pass if you score above 7. This is a **competitive ranking**:

The system does NOT issue PASS/FAIL per attempt.

The system issues ONE SCORE per attempt.

The system maintains ONE RANKING.

PASS/FAIL is issued only at convocation CLOSURE,
based on final ranking + number of spots.

Think of it like the Olympics: getting 9.5 doesn't guarantee you a spot if everyone else scored 9.6.

The team

Person	Role	Language
Antonio		Spanish (English good)

Person	Role	Language
	Tech lead · Webfleet code · CMadrid client	
Jesús	Backend (Flask, models, scoring engine)	Spanish (English basic)
Alejandro	Frontend (Jinja2 templates, CSS, UX)	Spanish (English basic)
You (Joel)	Scoring Simulator · QA · E2E · Seed data · CI/CD	English

Daily standup: **09:30 sharp**. You speak last, in English, 3 sentences (what I did, what I'll do, what's blocking me).

The deadline

Monday May 11, 2026 — the CMadrid demo. Antonio travels to Madrid. The whole team has been working toward this for 14 days. There is no extension.

The Real Tech Stack

What is actually in the repository

The repository lives at <https://github.com/cosigein/training>. When you clone it, you will find:

```

training/
├─ app/                                ← Flask application (Python)
│   └─ __init__.py                    ← Factory: creates the Flask app
│   └─ config.py                      ← Dev / Testing / Production
configs
│   └─ extensions.py                  ← db, jwt, socketio, login, csrf,
limiter, ...
│   └─ blueprints/                    ← 12 blueprints (URL groups)

```

— admin/ goes here)	← /admin (your simulator endpoint
— auth/	← /auth
— manager/ dashboard)	← /manager (manager-facing
— sessions/ domain)	← /sessions (attempts – Jesús'
— uploads/	← /uploads
— ... (8 more)	
— models/	← SQLAlchemy models
— training.py domain)	← Convocatoria, Enrollment (core
— session.py	← Attempt (= one evaluation drive)
— auth.py	← User, Organization
— ...	
— middleware/	← audit.py, jwt_handlers.py
— services/ implemented)	← Mostly scaffolding (not yet
— sockets/	← Scaffolding (not yet implemented)
— workers/	← Celery workers (scaffolding)
— utils/	
— decorators.py	← @require_role([...]) – ONLY auth
check	
— static/css/ components/*.css	← tokens.css, reset.css,
— templates/	← Jinja2 HTML templates
— migrations/	← Alembic (database migrations)
— tests/	← pytest (conftest.py is empty –
you start here)	
— seed_geofences.py	← Example seed script
— wsgi.py	← Entry point for the dev server
— celery_worker.py	← Celery entry point

The stack, line by line

Layer	Technology
Language	Python 3.12
Web framework	Flask 3 with blueprints
Database ORM	SQLAlchemy 2.0 + Flask-SQLAlchemy
Database	PostgreSQL (with PostGIS for geofences)

Layer	Technology
Migrations	Alembic via Flask-Migrate
Auth	Flask-JWT-Extended (JWT in cookies) + Flask-Login
Session protection	Flask-WTF (CSRF)
Background jobs	Celery + Redis (broker)
Real-time	Flask-SocketIO + Redis
Templating	Jinja2 (server-side rendering — no React, no Vue)
Testing	pytest (unit/integration) + Playwright (E2E)
Observability	Loguru + Sentry

Running the project locally

1. Clone the repo

```
git clone https://github.com/cosigein/training.git
cd training
```

2. Create and activate virtual environment

```
python -m venv .venv
source .venv/bin/activate
```

3. Install dependencies

```
pip install -r requirements.txt -r requirements-dev.txt
```

4. Set up environment variables

```
cp .env.example .env
# Edit .env with your local DB credentials
```

5. Apply database migrations

```
flask db upgrade
```

6. Run the development server

```
python wsgi.py
```

The server will be available at <http://localhost:5000>.

The Scoring System — Explained from Scratch

Before you can build the simulator, you need to understand exactly what is being simulated. This section explains the scoring system completely.

How an attempt is evaluated

When a candidate drives the fire truck on a test route, the Doback Elite device (installed in the truck) captures raw sensor data. This data goes through a pipeline:

```
RAW SENSOR DATA
|
▼
NORMALIZATION (normalizerVersion)
| GPS coordinates cleaned
| IMU data filtered
| Data quality assessed
▼
EVENT DETECTION (detectorVersion)
| "Harsh brake detected at 14:32:01"
| "Route deviation at segment 3"
| "Speed limit exceeded on Calle Mayor"
▼
SCORING ENGINE (criteriaVersion)
| Applies weights to 4 families
| Computes score 0-10
▼
SCORE + BREAKDOWN
score = 7.4
scoreBreakdown = {
  "stability": 8.1,
  "speed": 6.8,
  "route": 7.2,
  "driving": 7.5
}
```

The 4 scoring families

The scoring formula has **4 families of variables**, each with a configurable weight:

Family	Default Weight	What it measures
Stability (estabilidad)	40%	IMU data: accelerometer, gyroscope, lateral G-forces, roll, pitch events
Speed (velocidad)	30%	GPS + Webfleet data vs road speed limits
Route (ruta)	15%	GPS track vs planned route, segment deviations, timing
Driving (conducción)	15%	Harsh braking, harsh acceleration, general behavior

The weights are configurable without recompiling. An admin can change them (e.g., increase stability to 50%, decrease speed to 20%). **But the sum must always be 100%.**

The final score is a **continuous number from 0.0 to 10.0** (not a letter grade, not a category).

CriteriaVersion — the versioning system

Every time the scoring weights or thresholds change, a new CriteriaVersion is created. This is critical for legal correctness.

Simplified: what a CriteriaVersion looks like

```
{
  "id": "criteria-v2.1",
  "pesoEstabilidad": 0.40,
  "pesoVelocidad": 0.30,
  "pesoRuta": 0.15,
  "pesoConduccion": 0.15,
  "umbralFrenazoFuerte": 0.8,      # threshold for "harsh
    brake" detection
  "umbralDesviacionRuta": 50,      # meters deviation before
    penalty
  "umbralExcesoVelocidad": 5,      # km/h over limit before
    scoring drop
  # ... 11 more configurable parameters
}
```

When an Attempt is **created** (opened), the system takes a snapshot of the currently active CriteriaVersion and pins it to that attempt. **This version never changes for that attempt**, even if an admin updates the criteria the next day.

This is **Invariant #4** (explained fully in the next section).

What the Convocatoria model stores

The Convocatoria (exam call) model in app/models/training.py contains:

```
class Convocatoria(db.Model):
    name = "2026-DEM0"
    plazas = 10                # number of spots available
    umbralMin = 5.0            # minimum score (0-10) to be
                              # eligible
    pesosPorFamilia = {        # overridable weights
        "estabilidad": 0.40,
        "velocidad": 0.30,
        "ruta": 0.15,
        "conduccion": 0.15
    }
    criteriaVersion = "criteria-v2.1" # pinned at convocation
                                     # open
    status = "OPEN"             # OPEN → CLOSING → CLOSED → LOCKED
```

What the Attempt model stores

The Attempt model in app/models/session.py contains:

```
class Attempt(db.Model):
    # Who drove what
    studentId = "uuid-of-candidate"
    convocatoriaId = "uuid-of-convocatoria"
    routeId = "route-urban-center"

    # When
    startTime = datetime(2026, 5, 3, 10, 0, 0)
    closedAt = datetime(2026, 5, 3, 10, 47, 23) # NULL until
                                                # closed

    # Status
    status = "CLOSED"
```



```
# Scoring (populated when closed)
score = 7.4 # final score 0-10
scoreBreakdown = {
  "stability": 8.1,
  "speed": 6.8,
  "route": 7.2,
  "driving": 7.5
}

# Version pinning (captured at CREATE, never changed)
criteriaVersionPinned = "criteria-v2.1"
normalizerVersionPinned = "normalizer-v1.3"
detectorVersionPinned = "detector-v1.0"

# No `decision` field – PASS/FAIL does not exist per attempt
```

Notice: **there is no decision field** on an Attempt. There is no “PASS” or “FAIL” per drive. That decision only exists in the final CandidateOutcome created at convocation closure.

The 9 Invariants — Laws You Must Never Break

These 9 rules are architectural laws. If your code, a migration, or a PR would violate any of these, it gets rejected. Period.

Invariant 1 — Idempotency

Any endpoint or job can be called N times with the same input and produce the same result. No cumulative side effects.

Why it matters to you: Your seed script can be re-run without duplicating data. Your E2E tests can be run multiple times without corrupting state.

Invariant 2 — Reproducibility

Given the same input + same criteria + same dataset, the scoring output is **bit-for-bit identical**, today or in 5 years.

Why it matters to you: Your simulator endpoint must call Jesús' `simulate()` pure function — a function with no side effects, no randomness, no timestamp dependencies.

Invariant 3 — Closed attempt immutability

A closed Attempt is never modified. Not the score, not the timestamps, not the metadata. If a reevaluation is needed, a new Attempt is created referencing the old one.

Why it matters to you: Your simulator endpoint must compute new scores **in memory only** — never write to any Attempt row. If a database write happens, Antonio rejects the PR.

Invariant 4 — CriteriaVersion pinned at CREATE

When an Attempt is created (opened), the active `criteriaVersion` is pinned to it. Changing criteria after that does not affect the attempt.

Why it matters to you: Your E2E test #7 (simulator) must verify that after a criteria override, the **original** attempts still show their **original** scores. The simulated ranking lives only in the API response.

Invariant 5 — Source and confidence are orthogonal

Every data point has two independent metadata fields:

- `source`: where it came from (`sensor`, `webfleet`, `manual`, `gps`)
- `confidence`: how reliable it is (0.0 – 1.0)

These are independent. A `webfleet` reading can have `confidence = 0.9` if corroborated by GPS, or `confidence = 0.3` if it's the only source.

Why it matters to you: Your sensor fixture files must include both fields on every event.

Invariant 6 — Final ranking immutability

Once the final closing ranking is computed (`finalRankingSnapshot`), it cannot be recalculated. Late data is ignored.

Why it matters to you: Your E2E test #10 must verify that a CLOSED convocation's `finalRankingSnapshot` field cannot be overwritten.

Invariant 7 — Decision only at closure

The system **never** outputs PASS/FAIL per attempt. This decision only appears in CandidateOutcome records created during the 3-step closing process.

Why it matters to you: Your simulator endpoint response must **never** contain a decision or apto field per attempt. Only scores and ranking positions. Antonio will reject any response that contains this.

Invariant 8 — Dual admin validation + SHA256 acta

Convocation closure requires two distinct administrators. The closing admin (#1) cannot be the confirming admin (#2). The act (acta PDF) is SHA256-signed for cryptographic proof.

Why it matters to you: Your E2E test #10 must verify that calling /close/confirm with the same account that initiated closure returns **403 Forbidden**.

Invariant 9 — CriteriaVersion pinned at CREATE, not at evaluate

Reinforcing Invariant #4: the pin happens at attempt **creation** (opening), not at evaluation time. This eliminates the race condition where criteria could change between opening and evaluating an attempt.

Why it matters to you: In your seed data, attempts created under criteria-v2.1 must keep criteria-v2.1 pinned even if you later add a criteria-v3.0 to the database.

Your Role — What You Build

You own **5 things** in this sprint:

1. **The Scoring Simulator (D12)** — your main deliverable
2. **E2E Tests (Playwright)** — 10 tests, all critical flows
3. **CI/CD pipeline** — GitHub Actions (tests on every PR, deploy on merge)
4. **Seed data** — realistic demo data (50 candidates, realistic scores)
5. **DEMO-READINESS.md** — weekly status doc (Friday updates, in Spanish)

What you do **not** own:

- The pure simulate() function — Jesús delivers this end of Day 9
- The base Jinja2 layout and CSS components — Alejandro

- Webfleet integration — Antonio
 - Domain models and scoring engine — Jesús
-

The Scoring Simulator (D12) — Your Main Deliverable

What the simulator does

The scoring simulator lets an admin ask: “*What would happen to the ranking if I changed these scoring weights?*”

The admin opens the simulator screen, types override values (e.g., “increase stability weight to 50%, decrease speed to 20%”), hits “Simulate”, and sees:

- The **original ranking** (based on current criteria)
- The **simulated ranking** (based on override criteria, computed in memory)
- A **diff**: which candidates crossed the cut-off (were outside top-10 and are now inside, or vice versa)

Nothing in the database changes. The simulation is read-only and lives only in the HTTP response.

What you write

1. The endpoint: **POST /admin/scoring/simulate**

This goes in `app/blueprints/admin/routes.py`.

Request body:

```
{
  "convocatoriaId": "uuid-of-convocatoria",
  "criteriaOverride": {
    "pesoEstabilidad": 0.50,
    "pesoVelocidad": 0.20,
    "pesoRuta": 0.15,
    "pesoConduccion": 0.15
  }
}
```

```
}  
}
```

Response:

```
{  
  "convocatoriaId": "uuid-of-convocatoria",  
  "rankingOriginal": [  
    {"rank": 1, "studentId": "...", "studentName": "Ana  
    García", "score": 8.9},  
    {"rank": 2, "studentId": "...", "studentName": "Luis  
    Pérez", "score": 8.4},  
    ...  
  ],  
  "rankingSimulado": [  
    {"rank": 1, "studentId": "...", "studentName": "Luis  
    Pérez", "score": 8.7},  
    {"rank": 2, "studentId": "...", "studentName": "Ana  
    García", "score": 8.5},  
    ...  
  ],  
  "candidatesCrossingCutoff": {  
    "newlyInside": [  
      {"studentId": "...", "studentName": "María Torres",  
        "originalRank": 12, "simulatedRank": 9}  
    ],  
    "newlyOutside": [  
      {"studentId": "...", "studentName": "Carlos Ruiz",  
        "originalRank": 10, "simulatedRank": 13}  
    ]  
  },  
  "summary": {  
    "totalAttempts": 30,  
    "plazas": 10,  
    "criteriaVersionUsed": "criteria-v2.1",  
    "criteriaOverrideApplied": true  
  }  
}
```

Hard rules for the endpoint:

- Must be gated with `@require_role(['ADMIN'])` — managers cannot access this
- Must write one row to the audit log (every simulate call is tracked)
- Must call Jesús' `simulate()` pure function — don't reimplement scoring logic
- Must return `criteria_version` used in the summary

- Must **never** write to any Attempt row
- Must **never** return a decision or apto field per attempt
- Response time under 2 seconds for 100 attempts (it's all in-memory computation)

Where Antonio will review:

Open this as a separate PR and tag Antonio as a required reviewer. He checks:

- Invariants 3, 7, 9 are respected
- The audit log row is written
- The RBAC decorator is correct
- No production data is mutated

2. The simulator screen (Jinja2 template)

IMPORTANT: There is no React in this project. The UI is Jinja2 server-side rendering. Your screen is an HTML template, not a `.tsx` file.

The file goes in:

`app/blueprints/admin/templates/admin/simulator.html`

The screen has three parts:

1. **Override form** — inputs for each scoring family weight (must sum to 100%). Submit button labeled “Simulate”.
2. **Original ranking table** — shows current ranking with scores per candidate.
3. **Simulated ranking table** — shows new ranking after override, with visual diff highlighting who crossed the cut-off.

Alejandro provides the base layout components (CSS classes, the `base.html` template to extend). You build the form and result tables on top of that.

A `data-testid` attribute is required on every interactive element so your Playwright tests can target them:

```
<input data-testid="admin-simulator-weight-stability" ...>
<button data-testid="admin-simulator-submit" ...>Simulate</button>
<table data-testid="admin-simulator-ranking-original" ...>
<table data-testid="admin-simulator-ranking-simulated" ...>
```

The naming convention is `<portal>--<screen>--<element>`. Ask Alejandro for confirmation on the portal prefix.

3. User guide: docs/SIMULATOR-USER-GUIDE.md

A 1-2 page document in **formal Spanish** (not Rioplatense — the CMadrid admin reads this). Step-by-step:

1. How to open the simulator screen
 2. What an “override” means (you’re testing a hypothesis, not changing official criteria)
 3. How to read the ranking diff
 4. What “crossing the cut-off” means in the competitive exam model
 5. A clear warning that simulations do not affect official scores
-

E2E Tests (Playwright)

Setup

Your Playwright tests live in `tests/e2e/`. The test framework is **Python’s `pytest-playwright`** (not Node’s `@playwright/test` — remember, this is a Python project).

Install:

```
pip install pytest-playwright
playwright install
```

Configuration file: `tests/e2e/conftest.py` (you create this).

The data-testid convention for selectors: `<portal>--<screen>--<element>`.

The 10 tests you must deliver

Test 1 — 3-role login (Day 3)

Verify that admin, manager, and candidate users can each log in and reach their respective dashboards.

Given an existing admin/manager/candidate user

When they navigate to `/login` and submit credentials

Then they reach the correct dashboard

And the correct navigation items are visible for their role

Test 2 — Sensor file upload creates samples (Day 7)

Given an attempt created for a candidate
When a manager uploads a valid sensor TXT file
Then the attempt has more than 0 sensor readings
And uploading the SAME file a second time does NOT create duplicate readings
(this verifies Invariant #1: idempotency)

Test 3 — Webfleet sync brings events (Day 7)

Given an attempt with sensor data
When a mocked Webfleet sync runs
Then the attempt has events with source="webfleet"
And every event has both source and confidence assigned
(this verifies Invariant #5: source + confidence orthogonal)

Test 4 — Closing an attempt produces a score (Day 8)

Given an attempt with sensor data and events
When a manager closes the attempt
Then attempt.closedAt is populated
And attempt.score is between 0 and 10
And attempt.status is "CLOSED"
And there is NO `decision` field on the attempt
(this verifies Invariant #7: decision only at closure)
And the attempt appears in the manager's evaluation matrix

Test 5 — Ranking computes correctly (Day 9)

Given N closed attempts in a convocation
When the daily ranking computation runs
Then a ranking snapshot exists with N entries
And entries are ordered correctly (highest score = rank 1)
And the manager can see the ranking on the manager dashboard

Test 6 — Audit request + resolution (Day 10)

Given a closed attempt with a low score
When the candidate submits an audit request with reason >= 30 characters
Then the request appears in the manager's audit queue

When the manager resolves with "Create reevaluation"
Then a new Attempt exists with parent_attempt_id pointing to the original
And the original attempt is unchanged (closedAt and score intact)
(this verifies Invariant #3: closed attempt immutability)

When the manager resolves ANOTHER audit with "Reject"
Then that AuditRequest has status REJECTED

Test 7 — Simulator returns correct diff (Day 11)

Given 30 closed attempts in a convocation with 10 spots
When an admin opens the simulator and submits with a threshold override
Then the response contains both rankingOriginal and rankingSimulado
And candidates_crossing_cutoff identifies who moved in/out
And NOTHING in the database changed (original attempts keep original scores)
(this verifies Invariants #3 and #7)

Test 8 — Kiosk opens and closes an attempt via RFID (Day 11)

Given a paired kiosk and two RFID cards (A and B)
When RFID-A is tapped
Then the kiosk shows the active attempt screen with candidate A's info
And an Attempt is created in the backend

When RFID-B is tapped after 5 seconds
Then the first attempt closes (status INTERRUPTED_BY_OTHER_CARD)
And a new attempt opens for candidate B

Test 9 — Kiosk recovery after reload (Day 13)

Given a kiosk with an attempt in progress
When the browser is reloaded (F5 simulated)
Then a recovery modal appears (K4 screen)
And the candidate can choose to continue or close the attempt

Test 10 — Convocation closing requires dual admin (Day 11+)

Given an OPEN convocation with 30 closed attempts and 10 spots

When admin#1 calls POST /close/preview

Then they see the simulated final ranking

When admin#1 calls POST /close/initiate

Then the convocation moves to CLOSING status

When admin#1 tries POST /close/confirm with their OWN account

Then they receive 403 Forbidden

(this verifies Invariant #8: dual admin validation)

When admin#2 (different user) calls POST /close/confirm

Then the convocation moves to CLOSED

And a finalRankingSnapshot exists with N entries

And N CandidateOutcome rows exist with decision PASS or FAIL

And reversal_window_until = closed_at + 24 hours

Seed Data — Making the Demo Credible

The demo is the moment CMadrid decides whether to buy. The data must look real. Not “Test User 01”. Realistic Spanish names, plausible score distributions, clear narratives.

The demo cohort

Create these in `seed_demo.py` (a Python script, similar to `seed_geofences.py`):

One convocation

```
convocatoria = Convocatoria(  
    name="2026-DEMO",  
    plazas=10,  
    umbralMin=5.0,  
    pesosPorFamilia={"estabilidad": 0.40, "velocidad": 0.30,  
                     "ruta": 0.15, "conduccion": 0.15},  
    criteriaVersion="criteria-v2.1",
```

```

        status=ConvocatoriaStatus.OPEN
    )

# User accounts required for /health/deep to pass
users = [
    {"email": "supervisor@cmadrid.es", "role": "SUPER_ADMIN"},
    {"email": "admin1@cmadrid.es",      "role": "ADMIN"},
    {"email": "admin2@cmadrid.es",      "role": "ADMIN"},
    # ... 50 candidates with realistic names
]

```

Score distribution for demo impact

Design the scores so that changing one threshold (e.g., stability weight from 40% to 50%) moves exactly **4-5 candidates** across the 10-spot cut-off line. This is the “wow moment” of the simulator demo.

Candidates 1–8: scores 8.5 – 9.5 (safely inside, won't move)
 Candidates 9–10: scores 7.8 – 8.1 (just inside – will move with override)
 Candidates 11–13: scores 7.5 – 7.7 (just outside – will move with override)
 Candidates 14–50: scores 3.0 – 7.4 (safely outside)

Required scenarios for the demo

Scenario	What it shows
1 candidate with 3 consecutive low scores	Alert indicator (⚠) in the matrix
1 candidate with clearly improving scores	Progress narrative
1 candidate with highest scores (top 3)	Strong contender visual
1 candidate with a reevaluation that improved their score	Audit flow working
1 candidate who didn't complete all routes	Ranking penalty visible

Fixture files

Create these files in tests/fixtures/:

```
tests/fixtures/
├── sensor/
│   ├── clean_drive_high_score.txt      ← GPS + IMU data → score
│   │   ~8.5
│   ├── harsh_braking_mid_score.txt    ← braking events → score
│   │   ~6.0
│   ├── route_deviation_low_score.txt  ← off-route sections →
│   │   score ~4.5
│   ├── data_gaps_medium_quality.txt   ← GPS gaps → quality
│   │   MEDIUM
│   ├── many_gaps_low_quality.txt      ← many gaps → quality
│   │   LOW
│   └── webfleet_backup_coverage.txt    ← Webfleet fills GPS gap
├── webfleet/
│   ├── normal_sync.json               ← standard Webfleet
│   │   payload
│   ├── down_partial.json              ← Webfleet unavailable,
│   │   GPS covers
│   └── ...
└── simulation/
    ├── criteria_v2_1.json              ← base criteria
    ├── override_stability_up.json      ← +10% stability, -10%
    │   speed
    └── override_speed_up.json          ← +10% speed, -10%
    stability
```

CI/CD Pipeline

What CI does on every pull request

Create .github/workflows/ci.yml:

name: CI

on:

```
pull_request:
push:
  branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:17
        env:
          POSTGRES_DB: training_test
          POSTGRES_USER: training
          POSTGRES_PASSWORD: training_test
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
      redis:
        image: redis:7
        ports:
          - 6379:6379

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: pip install -r requirements.txt -r requirements-dev.txt

      - name: Run database migrations
        run: flask db upgrade
        env:
          DATABASE_URL: postgresql://
            training:training_test@localhost:5432/training_test
          FLASK_ENV: testing
```

- name: Run unit tests
run: pytest tests/ -v --ignore=tests/e2e
- name: Install Playwright
run: playwright install --with-deps chromium
- name: Run E2E tests
run: pytest tests/e2e/ -v
env:
 DATABASE_URL: postgresql://
 training:training_test@localhost:5432/training_test
 FLASK_ENV: testing

What CI does on merge to main

Create `.github/workflows/deploy-staging.yml`:

```
name: Deploy to Staging

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to staging VPS
        run: ./scripts/deploy-staging.sh
        env:
          STAGING_SSH_KEY: ${ secrets.STAGING_SSH_KEY }
          STAGING_HOST: ${ secrets.STAGING_HOST }
      - name: Run smoke tests on staging
        run: pytest tests/smoke/ -v
```

Smoke tests (post-deploy)

Create `tests/smoke/test_health.py`:

```
def test_health_endpoint(client):
    response = client.get("/health")
    assert response.status_code == 200

def test_deep_health_endpoint(client):
    response = client.get("/health/deep")
    assert response.status_code == 200
    data = response.get_json()
    assert data["db"] == "ok"
    assert data["redis"] == "ok"
```

Day-by-Day Calendar

Week 1

Day	Date	Your tasks
Day 1	Tue 28/04	Kickoff. Bootstrap pytest + Playwright. First smoke test. Open PR: chore(qa): bootstrap pytest and playwright.
Day 2	Wed 29/04	CI hardened. ci.yml running on every PR. Initial seed: 1 convocation, 5 candidates, 2 routes.
Day 3	Thu 30/04	E2E Test #1: 3-role login (once auth lands from Jesús).
Day 4	Fri 01/05	HOLIDAY — Spanish Labour Day. No work.

Week 2

Day	Date	Your tasks
Day 7	Mon 04/05	Seed v2: 10 candidates, 4 routes, 5 empty attempts. E2E Test #2 (upload). E2E Test #3 (Webfleet mock).
Day 8	Tue 05/05	

Day	Date	Your tasks
		E2E Test #4: close attempt → score. Seed v3: convocation with 30 closed attempts.
Day 9	Wed 06/05	E2E Test #5: ranking. Seed v4: score distribution for simulator demo. Coordinate with Jesús for <code>simulate()</code> signature.
Day 10	Thu 07/05	Morning: endpoint POST /admin/scoring/simulate — open PR, tag Antonio. Afternoon: template admin/simulator.html with Alejandro's components. E2E Test #6 (audit).
Day 11	Fri 08/05	Morning: write <code>SIMULATOR-USER-GUIDE.md</code> . E2E Test #7 (simulator). Fix any bugs. Afternoon: E2E Test #8 (kiosk). Full check of Annex C. Update <code>DEMO-READINESS.md</code> . Internal demo 18:00.
Day 12	Sat 09/05	Kiosk Torture — you lead. Whole team. 4-6 hours.
Day 13	Sun 10/05	Demo rehearsal. 3 full passes. Write plan B (pre-recorded screencast fallback).
Day 14	Mon 11/05	CMadrid demo.

Working with the Rest of the Team

Your interface with Jesús

- **End of Day 9:** Jesús delivers `simulate()` pure function. You need to know:
 - What module/file it will be in (ask him: likely `app/services/scoring.py` or similar)
 - Its exact signature: `simulate(attempts: list[Attempt], criteria_override: dict) -> list[SimulatedAttempt]`
 - Whether it returns new score objects or just updated scores

- If he changes a model or endpoint shape that your E2E tests depend on, he must warn you in advance

Your interface with Alejandro

- He provides the base Jinja2 template structure (layout, CSS classes, the `base.html` to extend)
- He adds `data-testid` attributes to screens he builds — if he changes one, he must warn you
- For D12 (the simulator screen), you're building on top of his components

RBAC — How authentication works

The system has two roles: ADMIN and MANAGER. There is one decorator that enforces this:

```
# In app/utils/decorators.py
@require_role(['ADMIN'])
def simulate():
    # only admins can reach this
    ...
```

Rules you must follow:

- Every route you add must have `@require_role(...)` explicitly
 - The decorator already calls `jwt_required()` internally — don't add it again
 - Manager cannot access the simulator (admin-only endpoint)
 - If you add a navigation link visible to managers, the backing endpoint must also allow managers — they must always be in sync
-

Definition of Done

Your work is complete when all of the following are true:

- ✓ Simulator returns correct ranking diff in < 2 seconds for 100 attempts
- ✓ Simulator screen (D12) shows ranking reordering with cut-off diff highlighted
- ✓ User guide for CMadrid is written (1–2 pages, formal Spanish)
- ✓ 10 E2E tests listed in this doc are passing in CI (not just

locally)

- ✓ Seed data: 50 realistic candidates, correct score distribution
 - ✓ Seed includes 1 SUPER_ADMIN + 2 distinct ADMIN accounts
(without this, /health/deep fails and the demo doesn't start)
 - ✓ CI runs on every PR: migrations + unit tests + E2E
 - ✓ Auto-deploy to staging on every merge to main
 - ✓ Smoke tests run post-deploy
 - ✓ DEMO-READINESS.md updated every Friday (in Spanish)
 - ✓ Day 12 (kiosk torture) executed; failures documented
 - ✓ Zero "tests pass locally but fail in CI" situations
 - ✓ Critical-flow E2E coverage $\geq 90\%$
-

Commit and PR Conventions

Branch naming

feat/qa-<description>	← new features you own
fix/qa-<description>	← bug fixes
chore/<description>	← setup, tooling, deps
test/<description>	← tests only

Commit messages (conventional commits — mandatory)

feat(qa): add Playwright smoke test for /health endpoint
fix(qa): correct data-testid selector in kiosk test
test(e2e): add E2E test #7 for scoring simulator
chore(ci): configure GitHub Actions for Python 3.12
seed: add demo cohort with 50 candidates and score distribution

Never: "fix things", "wip", "testing", "a". These will be rejected.

PR rules

- Every PR needs **1 review** minimum

- PRs that touch migrations/, app/utils/decorators.py, app/__init__.py, or any scoring/closing endpoint need **Antonio as mandatory reviewer**

- **Squash merge** by default

- CI must be green before merge

- PR body template:

```
## What changes
## Why
## How to test
## Risks
```

Glossary

Spanish (in code)	English	Notes
Convocatoria	Exam call / cohort	The overall process. Has N spots (plazas).
Attempt	Attempt	One evaluation drive. Has a score.
Enrollment	Enrollment	A candidate's registration in one convocatoria.
alumno / Student	Candidate	The human being evaluated. Model is called User with role CANDIDATE.
plazas	Spots	Fixed number of available seats.
ranking	Ranking	Competitive ordering. Updated daily.
cierre	Closing	3-step admin process to finalize a convocatoria.

Spanish (in code)	English	Notes
acta	Closing act / minutes	SHA256-signed PDF generated at closure. Legal document.
auditoría	Audit request	Formal challenge by a candidate to their score.
reevaluación	Reevaluation	New attempt created after a confirmed audit request.
Doback Elite	Doback Elite	Physical device in the truck. Captures sensor + GPS.
Webfleet	Webfleet	Bridgestone's fleet platform. CMadrid has it contracted.
kiosko	Kiosk	The in-cabin tablet where candidates tap RFID.
APTO / NO_APTO	PASS / FAIL	Only appears in CandidateOutcome at closure. Never on Attempt.
criteriaVersion	Criteria version	Versioned scoring weights. Pinned at attempt create.
blueprint	Blueprint	Flask's term for a URL group (like a router in Express).

Final Notes

The simulator is the centerpiece of the May 11 demo. When you show the CMadrid admin typing new weights and watching the ranking reorder live, identifying exactly who crosses the 10-spot line — that's the moment they decide to buy.

Build it well. Test it thoroughly. The data needs to look real. The response needs to be fast. The invariants need to hold.

If something is blocking you for more than 30 minutes, post in the team channel.
Antonio reads it constantly.

Good luck. Glad you're on the team.

— **Antonio**